

Instalación de WSL 2, Docker Desktop y PHP + Apache

En esta guía vamos a preparar el entorno de trabajo que utilizaremos durante el módulo de **Desarrollo Web en Entorno Servidor (DWES)**.

El objetivo final será disponer de:

- WSL 2
- Ubuntu
- Docker Desktop
- Docker Compose
- Un contenedor con Apache + PHP

INSTALACIÓN WSL

1. Comprobar el estado de WSL

Abrimos **PowerShell** y ejecutamos:

wsl --status

A partir del resultado pueden darse diferentes situaciones.

1.1. WSL no está instalado

Si aparece un mensaje indicando que WSL no está instalado, por ejemplo:

El Subsistema de Windows para Linux no está instalado.

Para instalar, ejecute "wsl.exe --install".

debemos instalarlo.

Ejecutamos:

wsl --install

Windows instalará WSL y, normalmente, la distribución **Ubuntu**.

Si se solicita reiniciar el equipo, reiniciamos.

Después del reinicio se abrirá Ubuntu y tendremos que crear:

- Un nombre de usuario.
- Una contraseña.

Estos datos son los del usuario de Linux y no tienen por qué coincidir con los de Windows.

Una vez finalizada la instalación, volvemos a PowerShell y comprobamos:

wsl --status

y:

wsl -l -v

Debemos comprobar que Ubuntu utiliza **WSL 2**:

```
NAME    STATE   VERSION
```

```
Ubuntu  Stopped  2
```

1.2. WSL ya está instalado y funciona correctamente

Si al ejecutar:

wsl --status

obtenemos información sobre WSL y comprobamos que la versión predeterminada es:

Versión predeterminada: 2

podemos continuar directamente con la instalación de Docker Desktop.

También podemos comprobar las distribuciones instaladas mediante:

wsl -l -v

Debemos comprobar que nuestra distribución, normalmente Ubuntu, aparece con:

```
VERSION
```

```
2
```

Por ejemplo:

```
NAME    STATE   VERSION
```

```
Ubuntu  Stopped  2
```

El estado Stopped es normal. Significa simplemente que Ubuntu no está ejecutándose en ese momento.

1.3. WSL aparece instalado, pero no funciona correctamente

En algunos equipos WSL puede aparecer instalado, pero al ejecutar:

wsl --status

puede aparecer un error.

Por ejemplo:

Clase no registrada

Código de error: Wsl/CallMsi/Install/REGDB_E_CLASSNOTREG

En este caso debemos comprobar si las características necesarias de Windows están habilitadas.

Abrimos **PowerShell como administrador**.

Comprobar "Subsistema de Windows para Linux"

Ejecutamos:

dism.exe /online /get-featureinfo /featurename:Microsoft-Windows-Subsystem-Linux

Buscamos:

Estado

Si aparece:

Estado: Deshabilitado

debemos habilitarlo:

dism.exe /online /enable-feature /featurename:Microsoft-Windows-Subsystem-Linux /all /norestart

Comprobar "Plataforma de máquina virtual"

Ejecutamos:

dism.exe /online /get-featureinfo /featurename:VirtualMachinePlatform

Si aparece:

Estado: Deshabilitado

debemos habilitarla:

```
dism.exe /online /enable-feature /featurename:VirtualMachinePlatform /all /norestart
```

Después de habilitar ambas características, **reiniciamos el equipo**.

Comprobar WSL después del reinicio

Volvemos a abrir PowerShell y ejecutamos:

```
wsl --status
```

Si WSL funciona correctamente, podemos continuar con la instalación/configuración de Ubuntu.

Si la instalación de WSL sigue dando problemas

Si WSL estaba instalado pero continúa dando errores, podemos realizar una reinstalación del paquete de WSL.

Este apartado solamente debe utilizarse si WSL presenta problemas. No es necesario realizarlo en una instalación normal.

En PowerShell ejecutamos:

```
Get-AppxPackage MicrosoftCorporationII.WindowsSubsystemForLinux |  
Remove-AppxPackage
```

Después reiniciamos Windows.

Al volver a iniciar el equipo, comprobamos:

```
wsl --status
```

Debería indicar que WSL no está instalado.

A continuación realizamos una instalación limpia:

```
wsl --install
```

Seguimos las instrucciones que aparezcan en pantalla y, si se solicita, creamos el usuario y contraseña de Ubuntu.

Finalmente comprobamos: **wsl --status**

y: **wsl -l -v**

Debemos comprobar que Ubuntu utiliza **WSL 2**.

INSTALACIÓN DE DOCKER DESKTOP

2.1. Instalar Docker Desktop

Una vez que WSL 2 funciona correctamente, podemos instalar Docker Desktop.

Descargamos Docker Desktop desde la página oficial:

<https://www.docker.com/products/docker-desktop/>

Durante la instalación utilizaremos **WSL 2** como backend cuando aparezca esta opción.

Una vez instalado, abrimos **Docker Desktop** y esperamos a que indique que Docker está funcionando.

2.2. Comprobar Docker

Abrimos PowerShell y ejecutamos:

docker --version

Después:

docker compose version

Ambos comandos deben mostrar la versión instalada.

A partir de ahora utilizaremos:

docker compose

y no el antiguo:

docker-compose

2.3. Crear nuestro primer proyecto Docker

Creamos la siguiente estructura:

C:\dwes

|

|— docker-compose.yml

|

|— src

|— index.php

2.3 Crear docker-compose.yml

Abrimos:

C:\dwes\docker-compose.yml

y escribimos:

services:

apache_php:

image: php:8.4-apache

volumes:

- ./src:/var/www/html

ports:

- "8080:80"

Este archivo indica a Docker cómo crear nuestro entorno de desarrollo.

Imagen

image: php:8.4-apache

Utilizaremos una imagen que ya contiene Apache y PHP 8.4.

Volumen

volumes:

- ./src:/var/www/html

Relacionamos nuestra carpeta de Windows:

C:\dwes\src

con la carpeta del contenedor:

/var/www/html

Por tanto:

Windows

Contenedor

C:\dwes\src —————→ /var/www/html

Puerto

ports:

- "8080:80"

Relacionamos:

Windows Contenedor

localhost:8080 —————→ Apache:80

Por eso accederemos a nuestra aplicación mediante:

http://localhost:8080

2.5. Crear nuestro primer archivo PHP

Abrimos:

C:\dwes\src\index.php

y escribimos:

```
<?php
```

```
echo "Hola desde Docker y PHP";
```

```
?>
```

Guardamos el archivo.

2.6. Arrancar el contenedor

Antes de arrancarlo debemos comprobar que **Docker Desktop está funcionando**.

Abrimos PowerShell y ejecutamos:

```
cd C:\dwes
```

Comprobamos el contenido:

```
dir
```

Debemos ver:

```
docker-compose.yml
```

```
src
```

A continuación ejecutamos:

docker compose up -d

Docker realizará las siguientes operaciones:

1. Leer el archivo docker-compose.yml.
2. Descargar la imagen php:8.4-apache si no está disponible.
3. Crear el contenedor.
4. Arrancar Apache y PHP.
5. Montar C:\dwes\src dentro del contenedor.

La primera ejecución puede tardar un poco porque Docker debe descargar la imagen.

2.7. Comprobar que PHP funciona

Abrimos el navegador y accedemos a:

http://localhost:8080

Si todo funciona correctamente, veremos:

Hola desde Docker y PHP

Esto significa que hemos conseguido ejecutar nuestro primer programa PHP dentro de un contenedor Docker.

2.8. Entorno de trabajo

La estructura final será:

Windows 11

|

|— WSL 2

| └─ Ubuntu

|

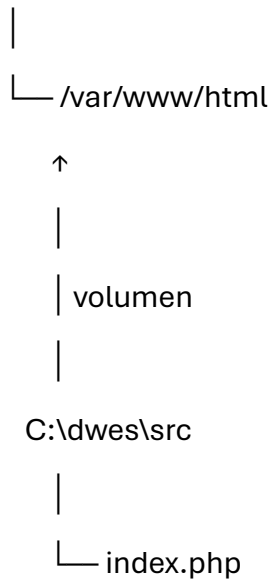
|— Docker Desktop

|

|— Contenedor

 |— Apache

 └─ PHP 8.4



Los archivos PHP se editarán desde Windows, por ejemplo utilizando **Visual Studio Code**, mientras que Apache y PHP se ejecutarán dentro del contenedor Docker.

Este será nuestro entorno de trabajo para las prácticas de DWES.

2.8. Comandos a utilizar

Inicia los servicios definidos en `docker-compose.yml`.

docker compose up -d

Detiene y elimina los contenedores creados por Compose.

docker compose ps

Muestra el estado de los servicios del proyecto.

docker compose ps

Muestra los mensajes generados por los servicios.

docker compose logs

2.9. ¿Dónde se ejecuta nuestro código?

Durante este módulo vamos a utilizar **Docker** para disponer de nuestro entorno de desarrollo.

Esto significa que no será necesario instalar Apache ni PHP directamente en Windows. Estos componentes estarán incluidos en el contenedor Docker que utilizaremos para ejecutar nuestras aplicaciones.

Nosotros trabajaremos principalmente con el código fuente de nuestras aplicaciones, que estará situado en la carpeta:

src

Docker se encargará de ejecutar **Apache y PHP dentro del contenedor** y de utilizar **nuestro código** almacenado en **src**.

De esta forma, podemos distinguir claramente entre:

- **Nuestro ordenador:** contiene nuestro proyecto y el código fuente.
- **Docker:** proporciona el entorno necesario para ejecutar la aplicación.
- **Contenedor:** ejecuta Apache y PHP.
- **Carpeta src:** contiene los archivos PHP que desarrollaremos durante el módulo.

Por ejemplo:

C:\dwes

```
|
|
|— docker-compose.yml
|
|
|— src
|   |
|   |— index.php
```

Cuando ejecutemos:

docker compose up -d

Docker iniciará el contenedor con Apache + PHP y nuestra aplicación estará disponible en el navegador.

Esta separación será especialmente útil más adelante, cuando trabajemos con Laravel, ya que podremos centrarnos en desarrollar la aplicación sin tener que configurar manualmente Apache y PHP en nuestro sistema.